Gota

O mordomo de TI que roda no seu computador

Documentação completa da versão Desktop (Windows): funcionalidades, como cada parte funciona, estrutura do projeto, tecnologias usadas e o que ele faz.

Repositório

`github.com/techemanuel21-creator/gota`

Este documento descreve o programa como ele está no código. Onde algo não foi testado ao vivo, isso está dito na seção "Limitações e pontos de atenção".

1. Visão geral

O **Gota** é um assistente pessoal com personalidade de gato mordomo. Ele conversa em português do Brasil, por texto ou por voz, entende o que está na tela do computador, abre programas, manda mensagens, lê e-mails e agenda, gera imagens, trabalha em projetos de código e responde como um especialista em TI (Veeam, VMware, Hyper-V, Nutanix, redes, segurança, Linux, Windows e desenvolvimento).

Existem dois projetos independentes no mesmo repositório: o aplicativo **Android** (pasta mobile/, Kotlin, com o mesmo modelo Gemma rodando no celular) e o programa **Windows** (pasta desktop/, Python), que é o assunto deste documento.

Princípios de projeto

- **Híbrido nuvem/offline:** online usa o modelo MiniMax pelo OpenRouter (rápido e mais capaz); offline, ou se a nuvem falhar, usa o Gemma que roda no próprio computador.
- **Dados no computador do usuário:** conversas, base de conhecimento, configurações e conexões ficam em %APPDATA%\Gota.
- **Pede permissão antes de agir:** ações com efeito (enviar mensagem, apagar, instalar, escrever fora do projeto) passam por confirmação; envio de WhatsApp mostra o texto para editar.
- **Sem truques:** o Gota não reescreve prompts do usuário às escondidas e não usa Acessibilidade nem controle remoto de aparelhos.
- **Aprende e se corrige:** quando não sabe algo, consulta a base de conhecimento, as habilidades e, se preciso, a documentação na internet.

2. Instalação e atualização

Instalação e atualização são feitas por comandos de uma linha no PowerShell, sem precisar de administrador.

```
# instalar
irm https://raw.githubusercontent.com/techemanuel21-creator/gota/main/desktop/install.ps1 | iex

# atualizar
irm https://raw.githubusercontent.com/techemanuel21-creator/gota/main/desktop/update.ps1 | iex
```

- O instalador usa **Python 3.12** (instala pelo winget se faltar), baixa o código do GitHub (zip), cria o ambiente virtual .venv, instala as bibliotecas e cria o atalho **Gota** na área de trabalho e no menu Iniciar. Fica em %LOCALAPPDATA%\Gota-App.
- A **atualização** fecha o Gota, **apaga o programa antigo por inteiro** e copia o novo. Mantém só os modelos de IA, o ambiente Python e os dados do usuário em %APPDATA%\Gota (conversas, conexões do WhatsApp e do Outlook, configurações). No fim mostra a **versão instalada** (código do commit).
- Os modelos de IA (alguns GB) são baixados no primeiro uso e guardados em models/. Dá para apontar outra pasta com a variável GOTA_MODELS.
- Também existe o empacotamento em **.exe** (PyInstaller, Gota.spec) com build.bat; o exe leva os modelos numa pasta ao lado.

Primeiro uso

- Pergunta como chamar o usuário (senhor, senhora ou só o nome).
- Pede a **chave de API do OpenRouter** (opcional). Sem chave, ou escolhendo "Usar só offline", o Gota usa só o Gemma local. A chave é guardada criptografada (DPAPI do Windows), nunca no código.
- Baixa e carrega o modelo local, o reconhecimento de voz e a voz.

3. Funcionalidades

3.1 Conversa por texto

- Janela própria sem moldura do Windows (barra de título, cantos arredondados, minimizar/maximizar), tema azul-marinho. Várias conversas, com título gerado automaticamente a partir da primeira mensagem, renomear e apagar.
- A resposta aparece **enquanto é escrita** (streaming). Blocos de código viram caixas separadas com botão **Copiar**. Textos são selecionáveis.
- Barra de atividade com o que o Gota está fazendo e botão **Parar**. O mesmo atalho de chamada cancela o que está em andamento.
- Anexar imagem à mensagem para o modelo analisar.
- Menu lateral: Nova conversa, Novo projeto, Selecionar conversa, Selecionar projeto, interruptor do "Hey Gota" e as janelas de configuração.

3.2 Voz

- **Atalho global Ctrl+Alt+G:** um gato desliza sobre a tela, escuta, responde falando e some. Apertar de novo cancela.
- **Ligação (Ctrl+Alt+L ou botão Ligar):** conversa contínua por voz numa janelinha pequena e transparente, com três botões (ver tela, mutar, desligar) e o gato com um telefone.
- **"Hey Gota":** escuta em segundo plano a palavra de chamada (opcional, interruptor no menu).
- **Ditado:** botão de microfone na caixa de mensagem escreve o que for dito.
- A **boca do gato** só mexe enquanto a voz realmente toca; fora disso ele fica de boca fechada e piscando.
- Voz masculina brasileira "faber" (Piper), mais lenta e digna. **Trechos em inglês** são falados por uma voz inglesa nativa (Piper "joe") ajustada ao tom e ao timbre da faber; nomes como JOB_BACKUP_TAPE_NOD01_B são soletrados ("job, backup, tape, nod zero um, bê"). Há um interruptor para desligar o inglês.
- **Microfone:** segue o padrão do Windows, relido a cada uso (acompanha o fone ao plugar), troca sozinho se um microfone ficar mudo, adapta taxa de amostragem e canais e permite escolher e testar o microfone na janela **Voz e microfone**.

3.3 Ver a tela e analisar

- "O que está na tela?", "analise o erro": o Gota fotografa a tela, **lê o texto com OCR** (Windows), procura na base de conhecimento e nas habilidades e responde **como especialista**: o que acontece, causa provável e a ação principal, de forma **resumida**, oferecendo os passos detalhados.
- Para listas (por exemplo, jobs do Veeam), detecta a **linha selecionada**, amplia e lê o nome exato, evitando inventar nomes.
- **Várias telas:** usa a numeração do Windows (Configurações > Tela). Entende "primeira tela", "tela 2", "outra tela", "as duas"; sem número, usa a tela da **janela ativa**. Em **Telas (apelidos)** o usuário clica em Identificar e dá apelidos ("notebook", "monitor grande"); o apelido tem prioridade.
- Se não conhece o erro e há internet, **procura e aprende a documentação** correspondente e guarda na

base.

3.4 WhatsApp

- Conexão feita **uma vez** dentro do Gota (QR code do WhatsApp Web num perfil privado do Edge). Sem conexão, o Gota diz que não está conectado.
- "Manda um print da tela 2 pro João" ou "manda pro Carlos que chego em 10 minutos": o texto é escrito **na voz do dono** (primeira pessoa), nunca como assistente, e uma janela mostra **para quem vai e o texto editável** antes de enviar. Para print, pergunta se quer mandar uma mensagem junto.
- O print vai como **foto JPEG** (pelo menu de anexo "Fotos e vídeos"), nunca como figurinha; pode enviar só uma parte da tela ("metade direita").
- A busca do contato tenta **grafias alternativas** (Emanuel/Emmanuel) e usa a grafia do nome do dono.

3.5 Outlook

- Login com a conta Microsoft (trabalho, escola ou pessoal) **dentro do Gota**, pelo fluxo de código de dispositivo (MSAL). Somente leitura: e-mails e agenda (permissões Mail.Read e Calendars.Read, via Microsoft Graph).
- "Qual a minha agenda de hoje?", "quais e-mails importantes preciso responder?". O token fica criptografado para o usuário do Windows.

3.6 Programas e computador

- Abre programas por nome, com **apelidos** definidos pelo usuário ("whats 2"). Catálogo dos programas instalados.
- Alarmes e temporizadores, clima, resposta imediata a contas, conversões e horas (sem usar o modelo), pesquisa na web (Bing, com reserva no DuckDuckGo).
- **Modo agente** para tarefas no computador: arquivos, pastas, relatórios, processos, rede, limpeza de disco, monitoramento, tarefas agendadas e mais (veja a seção 4.4).

3.7 Projetos e VS Code

- **Novo projeto, importar e selecionar projeto:** o Gota programa dentro da pasta do projeto, em passos, como um assistente de código: lê o repositório (mapa do repositório), edita arquivos com **backup automático**, roda testes e verifica o resultado. Uma permissão por sessão vale para a pasta do projeto.
- Painéis do projeto: **Arquivos** e **Tarefas em segundo plano**. Conversas e projetos são separados.
- Integração com o **VS Code**: extensão própria e um servidor local compatível com a API da OpenAI (127.0.0.1:11435, com token e sem aceitar origem de navegador). Se o VS Code não estiver instalado, o Gota oferece instalar, com permissão.

3.8 Imagens

- Gera imagens **offline** (modelo SDXS-512 DreamShaper pelo ExecuTorch, mais um decodificador de melhor qualidade). Salva em **Imagens > GOTA AI** no perfil do usuário.
- O modelo descreve o pedido, o pintor desenha e o resultado volta ao chat; "de novo" repete com o mesmo prompt.

3.9 Especialista em TI

- **Base de conhecimento** com 22 notas escritas (Veeam: erros, fita, arquitetura; VMware vSphere; Hyper-V e cluster; Nutanix; redes e Cisco IOS; segurança e resposta a incidentes; Windows Server e AD; Linux; armazenamento e RAID; backup e DR; virtualização e nuvem; método de troubleshooting; programação; engenharia de software).
- **23 habilidades** de diagnóstico passo a passo (por exemplo: job do Veeam falhou, "aborted due to server termination", fita falhou, host VMware desconectado, VM Hyper-V não inicia, saúde do cluster Nutanix, rede lenta, conta comprometida, resposta a ransomware, disco cheio, configurar portas e VLANs Cisco). O usuário pode criar as suas.
- **Download de documentações:** o botão "Baixar documentações" lê da internet 84 assuntos de TI e guarda na base do usuário (não faz parte do programa). Cada instalação baixa as suas.
- Quando a resposta não está na base, o Gota aprende pela internet (documentação oficial primeiro) e guarda.

3.10 Nuvem e offline

- **Online:** MiniMax minimax-m3.1-flash:free pelo OpenRouter, também para imagens (análise de tela). Respostas em geral em 3 a 6 segundos; pedidos longos (scripts) levam mais.
- **Offline ou falha:** Gemma 4 E2B (LiteRT-LM, CPU). Se a nuvem falhar, ela é ignorada por 45 segundos para não atrasar cada mensagem.
- Janela **Nuvem (OpenRouter)**: trocar/testar/remover a chave, trocar o modelo, ligar ou desligar a nuvem. Com a nuvem ligada, mensagens e prints analisados são enviados ao OpenRouter/MiniMax; offline tudo fica local.

3.11 Agente avançado

- **Em paralelo:** várias consultas de leitura no mesmo passo e até 4 sub-agentes ao mesmo tempo (explorar, verificar, planejar, revisar, redigir e clone, que herda o contexto da conversa). Todos só leem.
- **Modo plano:** em tarefas de vários passos o plano aparece numa janela e só executa depois de Executar; uma lista de tarefas ao vivo mostra o andamento.
- **Memória entre conversas:** quem é o usuário, como gosta das coisas, ambientes e referências, guardadas sozinhas (sem chamar o modelo) e editáveis na janela Memória e rotinas. A busca de conversas procura também no conteúdo.
- **Documentos enormes:** lê PDF, DOCX e texto de qualquer tamanho em partes ao mesmo tempo (com a nuvem) e responde ou resume.
- **Navegador geral:** abre sites, lê a estrutura da página, clica, digita (nunca senhas), rola, tira print, lê erros do console e requisições de rede e testa em tela de celular, tablet e desktop. Site novo e cliques de envio, compra ou apagar pedem permissão.
- **Terminal e arquivos:** PowerShell e Bash, notebooks Jupyter célula por célula, busca profunda na web.
- **GitHub e Git:** pull requests, issues e CI pelo gh; cópia isolada do projeto (worktree) para testes arriscados.
- **Páginas e documentos:** páginas HTML interativas, gráficos legíveis em tema claro e escuro, apresentações PowerPoint e servidor local com o navegador.
- **Rotinas e avisos:** tarefas que rodam sozinhas em horário fixo (só com consultas) e avisam por notificação do Windows ou no celular (ntfy).
- **MCP e habilidades prontas:** servidores MCP de terceiros em Integrações; instalação de habilidades de um endereço, com confirmação.
- **CLI e SDK:** python main.py cli e a classe Gota em gota.sdk.

3.12 Aparência das respostas

- Markdown nas respostas: títulos, listas com marcador, listas numeradas e aninhadas, negrito e código; menu lateral agrupado (Conexões, Conhecimento, Preferências, Avançado); cartão dos passos do agente com um símbolo por tipo de ferramenta.

4. Como tudo funciona

4.1 O caminho de uma pergunta

```
Usuário (texto ou voz)
   |
   ▼
 Brain.run()  ── atalhos sem IA: contas, horas, clima, agenda, abrir app, "pesquise..."
   |
   ▼
 Modelo (nuvem se online, senão Gemma) responde em JSON:  {"action": ..., "text": ...}
   |
   ▼
 Brain.act()  ── executa a ação escolhida
      reply · open_app · whatsapp / whatsapp_send · outlook · look_at_screen
      use_computer (agente) · ask_expert · generate_image · pesquisa na web ...
   |
   ▼
 Resposta  →  chat (streaming)  e/ou  voz (frase por frase)
```

- O **prompt do sistema** é compacto (cerca de 4 mil caracteres) com a lista de ações e exemplos. Cada mensagem recebe uma linha de contexto: data e hora, cidade, telas e a janela ativa.
- **Perguntas de TI com a nuvem:** uma só chamada, com o material da base (habilidade + nota) anexado ao pedido. Perguntas que dependem de fatos que mudam (notícias, preços, clima) usam pesquisa na web.
- **Com o modelo local (offline):** primeira passada sem "pensar" para decidir a ação; para perguntas de TI, passa por expert_answer, que busca na base e nas habilidades, aprende da internet se faltar e responde só com o material.
- Temperatura baixa (0,1) e regra contra invenção: o Gota deve dizer quando não sabe.

4.2 Voz: do atalho ao som

```
Ctrl+Alt+G
  → beep, gato desliza, microfone abre (Vosk em tempo real, ganho automático)
  → fim da frase (silêncio de 0,9 s) → Whisper "base" refina o texto (~1 s)
  → Brain.run(spoken=True) → resposta em streaming
  → SentenceFeeder entrega frase por frase ao Speaker
  → Speaker: sintetiza uma frase à frente da que toca (sem pausas), voz faber (PT)
     + trechos em inglês com a voz inglesa ajustada ao timbre da faber
  → o gato abre a boca só enquanto o som toca
```

- **Reconhecimento:** Vosk (modelo pequeno PT-BR) dá o texto parcial na hora e detecta o fim da frase; o faster-whisper refina a frase inteira com vocabulário de TI.
- **Fala:** Piper (ONNX) com correção do som "lh" (que a voz engolia) e fonemas de inglês para trechos em inglês; reprodução com sounddevice.
- **Microfone:** sounddevice com re-leitura da lista de dispositivos a cada uso, reserva para outros microfones e reamostragem para 16 kHz.

4.3 Tela e OCR

- EnumDisplayMonitors lista os monitores com a numeração do Windows e a posição de cada um; o programa é **per-monitor DPI aware**, então as coordenadas batem com os pixels reais.
- A captura usa **BitBlt** direto no retângulo do monitor (esconde as janelas do próprio Gota). "Todas as telas" captura a área virtual inteira.
- O **OCR** é o do Windows (WinRT) com a imagem ampliada 2x. As linhas com palavras de erro são usadas para procurar habilidades e notas. Se o OCR não leu o erro, o modelo de visão é pedido para copiar a mensagem.
- A resposta final é gerada com: pergunta, imagem, texto do OCR, linha selecionada e material da base.

4.4 O agente

O agente executa tarefas em vários passos em um laço do tipo **ReAct**: o modelo escolhe uma ferramenta em JSON, o programa executa, devolve a observação, e repete até concluir (limite de passos; 30 em modo projeto). Há um plano inicial, verificação do resultado e **subagentes** de papéis principal, projeto, explorar (somente leitura) e verificar. Tarefas podem rodar em **segundo plano**.

Grupo	Ferramentas
Arquivos	list_dir, read_file (PDF, Excel, Word), search_files, regex_search, write_file, edit_file (tolerante a espaços, com backup), move_files, rename_files, make_report, archive (zip), convert_image
Código e projeto	run_command (lista de comandos permitidos no projeto), git, run_tests, repo_map, vscode, sql_query, env_vars, http_request
Sistema	system_info, processes, network_info, disk_cleanup, monitor, schedule_task
Web e saber	web_search, web_page, kb_search, kb_learn, skill
Outros	delegate (subagentes em paralelo), calculate, translate, send_message, screen_look, speak_to_file, transcribe_audio
Extras	browser, read_document, run_bash, notebook_edit, github, git_worktree, make_html, make_chart, make_slides, serve_and_open, notify, routine, mcp, memory, chat_search, skill_install

- **Permissões:** ações perigosas pedem confirmação (ask). No modo projeto, uma permissão por sessão vale para a pasta do projeto. Tudo que é feito vai para um **registro de ações** (auditoria) consultável no menu.
- **Código:** o modelo escreve código em blocos de texto (não dentro de JSON), e o programa anexa ao arquivo; falhas repetidas (4) encerram com um resumo.

4.5 Base de conhecimento e habilidades

- Notas em Markdown em %APPDATA%\Gota\kb, pesquisadas por **BM25** (relevância por palavras, sem banco de dados). As 22 notas de fábrica ficam em gota/seed e são copiadas se faltarem.
- **Habilidades** são pastas com SKILL.md (nome, descrição, palavras-chave e passos). As de fábrica estão em gota/skills; as do usuário, em %APPDATA%\Gota\skills. A busca por habilidade também é BM25.
- **Aprender da internet:** o modelo escreve consultas em inglês; o programa procura, prioriza sites de documentação, lê as páginas, mantém só as que realmente tratam do assunto e guarda como notas. Há também leitura de README de pacotes (PyPI e npm).

4.6 WhatsApp e Outlook por dentro

- **WhatsApp:** o Playwright controla o Edge com perfil persistente em %APPDATA%\Gota\whatsapp. O código procura o contato na busca, abre a conversa, anexa a foto pelo menu "Fotos e vídeos" e envia. É um método **não oficial**: mudanças no WhatsApp Web podem quebrá-lo.
- **Outlook:** MSAL com cliente público da Microsoft (Graph Command Line Tools), cache do token cifrado com DPAPI em outlook.cache.

4.7 Janela e interface

- Interface em **Tkinter** com componentes próprios: janela sem moldura (cantos arredondados do Windows 11, botão na barra de tarefas), botões que mudam de cor suavemente, caixa de mensagem em cartão arredondado, balões de chat com blocos de código.
- O gato é feito de sprites (PNG) com animações: parado/piscando, ouvindo, lendo, pintando, falando e as versões de telefone.
- O chat rola sozinho até a mensagem mais nova e segue o fim enquanto a resposta cresce, até o usuário rolar para cima.
- A janela e os diálogos abrem no **monitor em que o app está** (e a tela cheia também).

5. Estrutura do projeto

```
projeto/
├── README.md
├── mobile/                 App Android (Kotlin), independente
└── desktop/                Programa Windows (Python)
    ├── main.py             entrada (DPI, identidade na barra de tarefas, autoteste)
    ├── install.ps1 / update.ps1 / run.bat / build.bat / Gota.spec
    ├── requirements.txt
    ├── gota/               o programa
    │   ├── seed/           22 notas de conhecimento de fábrica
    │   ├── skills/         23 habilidades de diagnóstico
    │   └── assets/         ícone e sprites do gato
    ├── vscode-extension/   extensão do VS Code (package.json, extension.js)
    ├── models/             modelos de IA (fora do git)
    └── tests/              testes unitários, de interface e ao vivo
```

Módulos do programa (desktop/gota)

Arquivo	Responsabilidade
app.py (≈2200 linhas)	Janela principal, diálogos, chat, chamada por voz, atalhos, menu lateral, telas de primeiro uso
brain.py (≈980)	Cérebro: roteamento, prompt, ações, análise de tela, WhatsApp/Outlook, especialista
agent.py (≈1120)	Agente de tarefas e de projetos: ferramentas, plano, subagentes, verificação, permissões
llm.py / cloud.py	Gemma local (LiteRT-LM); cliente OpenRouter e a classe Hybrid (nuvem ou local)
voice.py (≈730)	Escuta (Vosk, Whisper), fala (Piper), microfone, atalho global, "Hey Gota"
tools.py (≈600)	Pesquisa web, clima, apps, alarmes, monitores, captura de tela, OCR, helpers

Arquivo	Responsabilidade
kb.py / skills.py / docpack.py	Base de conhecimento (BM25), habilidades, download de documentações
whatsapp.py / outlook.py	Conexões: WhatsApp Web (Playwright) e Outlook (MSAL/Graph)
vscode.py / server.py / projects.py	Integração com VS Code, servidor local compatível com OpenAI, projetos
imagegen.py	Gerador de imagens offline (SDXS via ExecuTorch)
ext_tools.py / browser.py / mcp.py	Ferramentas extras do agente, navegador geral (Playwright) e cliente MCP
memory.py / routines.py / sdk.py	Memória entre conversas, rotinas automáticas, uso como biblioteca e linha de comando
widgets.py / chrome.py / overlay.py	Componentes visuais, janela sem moldura, gato que desliza sobre a tela
store.py / quick.py	Perfil, configurações e conversas; respostas imediatas sem IA

Onde ficam os dados do usuário (%APPDATA%\Gota)

Item	Conteúdo
settings.json, profile	Configurações e nome/tratamento do usuário
chats	Conversas e o chat de voz
kb/ e skills/	Notas aprendidas ou baixadas; habilidades do usuário
openrouter.key	Chave da nuvem, cifrada com DPAPI (só abre para este usuário do Windows)
outlook.cache	Token do Outlook, cifrado com DPAPI
whatsapp/	Perfil privado do Edge com a sessão do WhatsApp Web
backups/, attachments/	Cópias de arquivos editados pelo agente; imagens anexadas
audit / gota.log	Registro de ações do agente; log do programa

6. O que ele usa

Modelos de IA

Modelo	Para quê	Onde roda
MiniMax M3.1 Flash (free)	Conversa, análise de tela com imagem, especialista	Nuvem (OpenRouter), quando online
Gemma 4 E2B (LiteRT-LM)	Tudo, quando offline ou se a nuvem falhar	CPU, local (~2,5 GB)
SDXS-512 DreamShaper + VAE	Geração de imagens	CPU, local (ExecuTorch)
Piper "faber" pt-BR	Voz do Gota	Local (ONNX, ~60 MB)
Piper "joe" en-US	Trechos em inglês	Local (ONNX, ~60 MB)
Vosk PT-BR (pequeno)	Reconhecimento de fala em tempo real	Local (~50 MB)

Modelo	Para quê	Onde roda
Whisper base (faster-whisper)	Refinar o texto falado	Local (~140 MB)

Bibliotecas Python

Biblioteca	Uso
litert-lm, torch, executorch	Gemma local e gerador de imagens
piper-tts, sounddevice, vosk, faster-whisper, praat-parselmouth, numpy	Voz, escuta, ajuste de timbre
pystray, pyttsx3, pillow	Ícone na bandeja, voz do Windows de reserva, imagens e sprites
playwright	WhatsApp Web (com o Edge instalado no Windows)
msal	Login e token do Outlook (Microsoft Graph)
pypdf, openpyxl, python-docx, fpdf2	Ler e criar PDF, Excel e Word
psutil	Informações de processos e sistema
Tkinter, ctypes (Win32)	Interface, monitores, captura de tela, DPAPI, atalhos globais
Windows OCR (WinRT) via PowerShell	Leitura de texto da tela

Serviços externos

- **OpenRouter** (modelo online); **Bing / DuckDuckGo** (pesquisa); **Hugging Face** (download dos modelos); **PyPI/npm** (README de pacotes); **GitHub** (instalação e atualização); **WhatsApp Web** e **Microsoft Graph**.
- Tudo opcional além do modelo local: sem internet, o Gota continua conversando, falando, vendo a tela e usando a base de conhecimento.

Licenças: ponto de atenção para venda

O projeto foi pensado para poder ser vendido no futuro. A maior parte das bibliotecas tem licenças permissivas, mas **vários componentes têm condições próprias** que devem ser conferidas antes de comercializar: o modelo **Gemma** (termos de uso do Google), as bibliotecas de licença **GPL/LGPL** (por exemplo piper-tts, praat-parselmouth, pystray, fpdf2), as vozes e modelos (a voz "joe" é domínio público; a "faber" é MIT) e o uso do **OpenRouter/MiniMax** (termos de cada serviço). O WhatsApp Web não tem integração oficial. Esta lista é informativa e deve ser confirmada diretamente nas licenças e termos de cada projeto.

7. Testes e qualidade

- **58 testes unitários** (tests/test_core.py): tokenizador, ferramentas, conversas, streaming, matemática rápida, habilidades, servidor local, agente (código, edição, backup, permissão), base de conhecimento, leitura de telas e nomes, nuvem com reserva local, voz (idiomas, nomes soletrados, ajuste de timbre), boca do gato, habilidade Cisco, agente em paralelo e plano, memória, rotinas, navegador, MCP e ferramentas extras.
- **Testes de interface** (gui_*.py): abrem a janela real, enviam mensagens, abrem diálogos e salvam capturas para conferência.
- **Testes ao vivo** (live_*.py): rodam o modelo real em cenários de roteamento, telas, conexões, projetos e servidor.
- **Autoteste do exe** (executar o Gota.exe com o parâmetro selftest): confere IA local, microfone, base, habilidades, documentos, vozes, Outlook, WhatsApp (Edge), VS Code e imagem.
- Extensão do VS Code testada com o módulo vscode simulado (7 verificações).

```
# rodar os testes unitários
cd desktop
.venv\Scripts\python.exe -m unittest tests.test_core
```

8. Limitações e pontos de atenção

- **Voz:** a faber é treinada só em português; o inglês usa outra voz ajustada, que se aproxima da faber mas não é idêntica. A qualidade final depende do ouvido do usuário.
- **Modelo local (Gemma 2B):** limitado em código complexo e raciocínio longo; é a reserva para offline. A experiência plena é com a nuvem.
- **Nuvem gratuita:** a velocidade depende do OpenRouter; scripts longos podem levar mais de um minuto, e há limites de uso do plano gratuito.
- **Privacidade:** com a nuvem ligada, o texto e os prints analisados saem do computador. Para telas sensíveis, desligue a nuvem.
- **WhatsApp:** integração não oficial; a sessão pode expirar e pedir novo QR code; mudanças na página do WhatsApp podem exigir ajuste.
- **Análise de tela:** a detecção do item selecionado reconhece o azul claro de seleção do Windows; outros temas caem no OCR comum.
- **Documentações baixadas:** de 84 assuntos, 56 renderam texto aproveitável na última execução (os demais usam páginas com JavaScript ou não passaram no filtro de relevância).
- **Ainda não testado ao vivo em todos os cenários:** ligação por voz de ponta a ponta com fone, envio real no WhatsApp após a última correção, login do Outlook, extensão dentro do VS Code real, instalação do VS Code, janelas em dois monitores e a instalação completa por install.ps1 numa máquina limpa.

9. Comandos úteis

```
# abrir a partir do código
desktop\run.bat

# checar as peças do programa empacotado
Gota.exe selftest (parâmetro com dois hifens na frente): grava selftest.txt na pasta de dados

# gerar o exe
cd desktop
.venv\Scripts\python.exe -m PyInstaller Gota.spec --noconfirm --clean --distpath dist --workpath
build

# atalhos
Ctrl+Alt+G  chamar o Gota por voz     Ctrl+Alt+L  ligar / desligar a ligação
```